**Q) Can we create empty Java file, can we compile and execute it?**

Yes, but after compilation we do not have .class file. So we cannot execute.

**Example. java**

```
>javac Example.java
```

**Q) Can we create empty class, can we compile and execute it?**

Yes we can create and compile empty class. Compiler generates .class file with that class name. We cannot execute this class as it does not have main method. It leads to exception

**Example. java**

```
class A{

}
```

```
>javac Example.java
        |-> A.class
>java A
Exception ……
```

**Q) Is it mandatory that java file name and class name should be same?**

No, not always- file name can be user defined name.

**Q) When should the Java file name and class name be the same?**

If class is declared as public, file name should be the same as the public class name, else its name can be user defined.

**Example. Java**

```
class A{

}
```

**Example.java**

```
public class A{

}
```

**CE**: class A is public, should be declared in a file named A.java

**Q) In a single Java file how many classes can we define?**

We can define more than one class. The rule is, class names should be different.

**Q) If a java file has multiple classes what is the java file name?**

public class name. If there is no public class, java file name can be user defined.

**Q) In a java file, how many public classes can we define?**

Only one. We can define one public class and multiple non-public classes.

**Example. Java**

```
class A{
}

class B{
}
```

**Example.java**

```
public class A{
}

class B{
}
```

**A.java**

```
public class A{
}

class B{
}
```

**A.java**

```
public class A{
}

public class B{
}
```

**Q) If we compile multiple classes java file, how many .class files are generated by compiler?**
Compiler generates .class files as many class definitions as we have in that java file. Compiler generates .class file separately for every class with that class name.

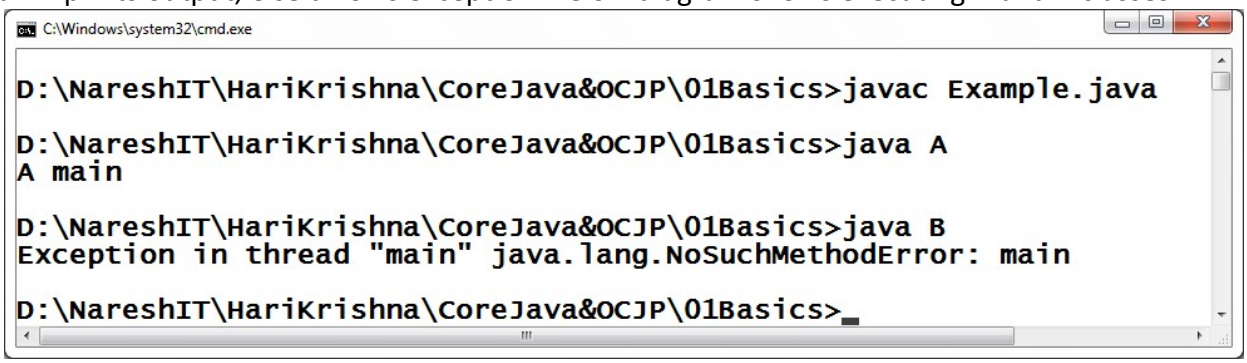
**Example.java**

```
class A{
        public static void main(String[] args){
                System.out.println("A main")
        }
}

class B{
}
```

```
>javac Example.java
        |-> A.class
        |-> B.class
```

**Q) How can we execute all classes?**
We should execute each class separately using Java command. If that class has main method JVM prints output, else throws exception. Below diagram shows executing *A* and *B* classes




**Q) What is meant by user defined method and predefined method?**
Developer defined method is called user defined or custom method.
Already defined methods are called predefined methods.
These methods may be developed by SUN Microsystem developers, or some other developers.
Ex: println() method is predefined.

**Q) What is meant by user defined class and predefined class?**
Developer defined class is called user defined or custom class.
Already defined class is called predefined class.
These class may be developed by SUN Microsystem developers, or some other developers.

For example : System, String are predefined classes
FirstProgram, A, B are user defined classes

**Q) Does JVM execute user defined methods automatically?**
No, it executes only main method. To execute user defined methods We must call them from main method or a method calling from main

**What is the output from below program?**

Example.java

```
class A{
        void m1(){
                System.out.println("A m1");
        }
        public static void main(String[] args){
                System.out.println("A main");
        }
}
```

```
>javac Example.java
>java A
A main
```

*m1()* method is user defined method. JVM has not executed it, because it is not called from main method.

Below program shows **calling m1()** method from **main method, *but*** leads to Compile time error: "**non-static method m1() cannot be referenced from static context**"

Example.java

```
class A{
        void m1(){
                System.out.println("A m1");
        }
        public static void main(String[] args){
                System.out.println("A main");

                m1();  ╳  CE: non-static method m1() cannot be referenced from static context
        }
}
```

The **Reason** for the above CE is m1() method does not have permission to access directly, for the time being assume it doesn't have memory.

JVM provides memory for variables and methods only if we use either of the below keywords

1. **static** or
2. **new**

In this chapter I only use static keyword, in next chapters I will teach you the usage of new keyword. Like in main method, for user defined methods also we must use static keyword before return type, then this method can be called directly from main method.

**Example.java**

```
class A{
        static void m1(){
                System.out.println("A m1");
        }
        public static void main(String[] args){
                System.out.println("A main");
                m1();
        }
}
```

```
>javac Example.java
>java A
A main
A m1
```

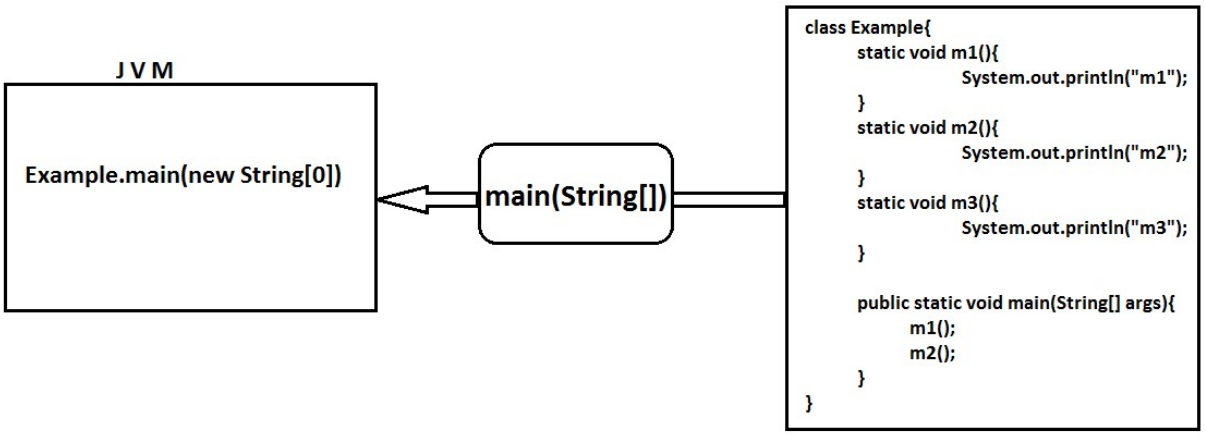
**Q) Does JVM execute user defined methods by itself just by declaring them as static?**
**A)** No, static methods also should be called from main method.

**Q) Then how main method is executed by JVM automatically? Is it not because of static keyword?**
**A)** Yes, main method is executed not because of static keyword, because it is called from JVM software.

**Why JVM executes only man method why it does not execute user defined methods?**
JVM executes only main method because it has main method's calling statement. It does not have user defined method's calling statement in its logic. It executes user defined methods only if they are called from main method.

**Why JVM software has only main method calling statement why not user defined methods? Or why main method is the initial point of class logic execution? Or Why JVM executes only main method why not user defined methods?**
JVM developer doesn't know what methods should be executed, when they should be executed and in which order.

Hence it is the developer responsibility to inform about the methods those must be executed by JVM. For this purpose there should be a common method to call these methods, that method should be known to both JVM and developer, and that method prototype should be given by the JVM S/W designer (vendor) as it must be called from JVM. That method is main method.




**Definition of main method**
main method is the mediator method between Java developer and JVM to inform what are the methods should be execution when and in which order.

**Q) How many user defined methods can we define in a class?**
We can define multiple methods, but rule is, method names should be different.

**Q) What is the order of execution of all user defined methods?**
In the order they are called from main method or from any one of the method calling from main method.

**What the output from below program?**

Example.java

```
class A{
        static void m1(){
                System.out.println("m1");
        }
        static void m2(){
                System.out.println("m2");
        }
        public static void main(String[] args){
                System.out.println("main");
                m1();
        }
}
```

```
>javac Example.java
>java A
main
m1
```

Example.java

```
class A{
        static void m1(){
                System.out.println("m1");
                m2();
        }
        static void m2(){
                System.out.println("m2");
        }
        public static void main(String[] args){
                System.out.println("main");
                m1();
        }
}
```

```
>javac Example.java
>java A
main
m1
m2
```

**Q) If a class does not have main method, how can we execute user defined methods of that class?**

We should use another class main method. We should call this method with its class name.

In projects, we do not write main method in every class. Instead we write main method in one class, and we will call all other classes' methods from this class's main method for testing.

Basically main method is given to start class logic execution but not for developing logic directly.

**Below program shows calling User defined method from another class MM**.

Example.java

```
class A{
        static void m1(){
                System.out.println("A m1");
        }
}

class B{
        static void m2(){
                System.out.println("B m2");
        }

        public static void main(String[] args)  {
                System.out.println("B main");
                m2();
                A.m1();
        }
}
```

```
>javac Example.java
>java A
Exception ……..

>java B
B main
B m2
A m1
```

**Q) Is it possible to create user defined method in all classes with same name?**
Yes, we can define user defined methods with same name in multiple classes. To call those methods we must use that method's class name.

**What is the output from the below program?**
Example.java

```
class A{
        static void m1(){
                System.out.println("A m1");
        }
}
class B{
        static void m1(){
                System.out.println("B m1");
        }
}
class Test
        public static void main(String[] args)  {
                System.out.println("Test main");
                A.m1();
                B.m1();
        }
}
```

```
>javac Example.java
>java Test
Test main
A m1
B m1
```

**Q) In single java file, can we define main method in all classes?**

Yes we can define. There is no CE, RE. Because compiler stores each class bytecode in separate .class file there is no confusion for JVM in executing main method.

**Check below program.**

```
//Example.java
class A{
        public static void main(String[] args) {
                System.out.println("A main");
        }
}

class B{
        public static void main(String[] args) {
                System.out.println("B main");
        }
}
```

```
>javac Example.java
>java A
A main

>java B
B main
```

**Q) Can we call main method explicitly?**

Yes, even the main method is a method, so we can call.

**Syntax to call main method**

**main(*new String[0]*);**

In place of ZERO we can pass any +ve integer number

**Write a program to call A class main method from B class main method.**

Example.java

```
class A{
        public static void main(String[] args) {
                System.out.println("A main");
        }
}

class B{
        public static void main(String[] args) {
                System.out.println("B main");
                A.main(new String[0]);
        }
}
```

```
>javac Example.java
>java A
A main

>java B
B main
A main
```

**Q) What does happen when we pass negative number as size to array object?**

For example: main ( new String[-5] );

A) No CE, but it leads exception "java.lang.NegativeArraySizeException"

**Q) What does happen when we call main method in the same class?**

```
class A{
        public static void main(String[] args){
                System.out.println("main");
                main( new String[0] );
        }
}
```

A) No CE, but it leads exception "java.lang.StackOverflowError", because it leads to recursive method call

**Q) What is the output from the below program?**

```
class A{
        public static void main(String[] args){
                System.out.println("A main");
                m1();
        }
        static void m1(){
                System.out.println("A m1");
                main( new String[0] );
        }
}
```

**Options:**

1. compile time error
2. exception
3. A main
   A m1
   A main
   A m1
   A main
   A m1
   exception

**Q) Can we overload main method?**

**A)** Yes

**Q) Then which main method is executed by JVM?**

**A)** String[] parameter method

What is the output from the below program?

```
class A{

   public static void main(String args){
      System.out.println("In main(String)");
   }

   public static void main(String[] args){
      System.out.println("In main(String[])");
   }

   public static void main(int[] args){
      System.out.println("In main(int[])");
   }
}
```

**Below program shows implementing all above points**
**//MultipleClasses.java**

```
class A{
}

class B{
	static void m1(){
		System.out.println("B m1");
	}
}

class C{
	static void m2(){
		System.out.println("C m2");
	}
	public static void main(String[] args){
		System.out.println("C main");
	}
}

class D{
	static void m3(){
		System.out.println("D m3");
	}
	public static void main(String[] args){
		System.out.println("D main");
		m3();
	}
	static void m4(){
		System.out.println("D m4");
	}
}
```

```
class E{
	static void m5(){
		System.out.pritln("E m5");
	}
	public static void main(String[] args){
		System.out.pritln("E main");

		m5();

		B.m1();
		C.m2();

		D.m3();
		D.m4();

		D.main(new String[0]);

		E.m5();
	}
}
```

**Compilation**

```
>javac MultipleClasses.java
		|->A.class
		|->B.class
		|->C.class
		|->D.class
		|->E.class
```

**Execution:**

**> java A**

Exception in thread "main" java.lang.NoSuchMethodError: main

**> java B**

Exception in thread "main" java.lang.NoSuchMethodError: main

**> java C**

C main

**> java D**

D main
D m3

**> java E**

E main
E m5
B m1
C m2
D m3
D m4
**D main**
**D m3**
E m5

**Q) How can we execute all class's methods with single class execution?**

A) Call all those methods from the current class main method or from the method that is calling from main method.

**"E" class is an example of above point development.**